

# NCCL, All-reduce, and Multi-node

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE  
Director, Centre for Artificial Intelligence Research,  
Prof., School of Computer Science & Engineering,  
KLE Technological University,  
BVB Campus, Vidyanagar,  
Hubballi - 580031, Karnataka, India

# Why Doesn't Comm Blow Up as GPUs Grow?

## Naive all-reduce

Every GPU sends its gradient to every other GPU.

With  $N$  GPUs and gradient size  $M$  bytes: per-GPU traffic =  $O(N \cdot M)$ .

Doubling GPUs DOUBLES per-GPU comm. This kills scaling at  $\sim 8$  GPUs.

## Ring all-reduce (what NCCL does)

GPUs arranged in a logical ring — each GPU only talks to its two neighbours.

Per-GPU traffic:  $2 \cdot (N-1) / N \cdot M$ .

As  $N \rightarrow \text{infinity}$ , traffic  $\rightarrow 2M$ . INDEPENDENT OF  $N$ .

**THE WIN:** per-GPU bandwidth is independent of cluster size. Training 8 GPUs costs the same comm per GPU as training 1024 GPUs — the algorithmic foundation of scaling to thousands of GPUs.

# Ring All-Reduce — The Algorithm

Split the gradient into  $N$  chunks. Two phases, each taking  $N-1$  steps: reduce-scatter, then all-gather.

## Phase 1: Reduce-scatter

Step  $k$ : each GPU sends chunk  $(\text{rank}-k) \bmod N$  to its right neighbour. Neighbour SUMS the incoming chunk with its own. After  $N-1$  steps, each GPU holds ONE fully-summed chunk.

## Phase 2: All-gather

Each GPU passes its fully-summed chunk around the ring.  $N-1$  hops, each receives the other  $N-1$  chunks. After  $N-1$  more steps, every GPU has the full sum.

Per-GPU bandwidth cost:  $2 * (N-1) / N * M$   
 (reduce-scatter sends  $M * (N-1) / N$ ; all-gather sends the same)

$N=2 \rightarrow 1.00 * M$  per GPU

$N=8 \rightarrow 1.75 * M$  per GPU

$\sim 2.00 * M$

$N=4 \rightarrow 1.50 * M$

$N=1024 \rightarrow$

# NCCL — Topology-Aware Implementation

## INTRA-NODE

**NVLink** — Direct GPU-to-GPU, 900 GB/s on H100

**NVSwitch** — Full-bandwidth all-to-all in HGX boxes

**PCIe P2P** — Direct GPU-GPU over PCIe, ~50 GB/s

## INTER-NODE

**InfiniBand + GPUDirect** — Zero-copy GPU-NIC. Best.

**RoCEv2 + GPUDirect** — RDMA over Ethernet, near-IB  
Remote Direct Memory Access

**Sockets (TCP)** — Fallback. 5-10x slower.

HOW NCCL PICKS: at init, NCCL probes `CUDA_VISIBLE_DEVICES`, reads NVML topology, discovers NICs. It builds a graph of which GPUs share NVLink, which share a PCIe root, which NICs are close to which GPUs — then picks the fastest ring that visits all ranks. You see the choice in `NCCL_DEBUG=INFO` output.

# Multi-node Rendezvous with Collective Communication (c10d)

## ( Pytorch distributed communications layer )

How do N independent processes across K machines find each other?

- 1 Each process starts knowing: its own rank, total `world_size`, a master `addr:port`.
- 2 Rank 0 opens a TCP store on `MASTER_ADDR:MASTER_PORT` and waits for connections.
- 3 All other ranks connect to the store and register themselves.
- 4 Once `world_size` processes have registered, they exchange a NCCL unique ID via the store.
- 5 NCCL uses the unique ID to set up peer-to-peer communicators between all ranks.
- 6 First collective runs. NCCL picks ring topology. Training starts.

WHEN IT FAILS: rendezvous timeout usually means a firewall blocks `MASTER_PORT`, wrong `MASTER_ADDR`, or ranks disagree on `world_size`. Check `'telnet node0 29500'` from each node first.

## NCCL Env Vars You'll Actually Use

| Var                       | Value                 | What                                                                |
|---------------------------|-----------------------|---------------------------------------------------------------------|
| NCCL_DEBUG                | INFO                  | Verbose logs. Set once during bring-up; off in production (spammy). |
| NCCL_SOCKET_IFNAME        | eth0 /<br>^lo,docker0 | Which network interface to use — essential with multiple NICs.      |
| NCCL_IB_HCA               | mlx5_0,mlx5_1         | Pin to specific IB HCAs. Prevents slow fallback.                    |
| NCCL_IB_DISABLE           | 1                     | Force TCP fallback — use to confirm IB is the culprit.              |
| NCCL_ASYNC_ERROR_HANDLING | 1                     | Errors become exceptions instead of deadlock. Highly recommended.   |
| NCCL_BUFFSIZE             | 8388608               | Bytes per collective chunk. Tune for small-message-heavy workloads. |

# The Bandwidth Ladder — Know Your Interconnect

Every hop away from the GPU die costs an order of magnitude. Knowing where your training job sits on this ladder explains most 'why is scaling worse than expected' questions.

**NVLink 5 (B200, intra-node)** ~1.8 TB/s

**NVSwitch (HGX, all-to-all)** ~900 GB/s

**InfiniBand NDR (inter-node)** ~400–800  
Gb/s

**RoCEv2 Ethernet (inter-node)** ~200–400  
Gb/s

**PCIe Gen5 P2P (fallback)** ~64 GB/s

**TCP sockets (last resort)** ~10–25  
Gb/s

## Session 3 Summary

- NCCL is topology-aware: NVLink/NVSwitch intra-node, InfiniBand/RoCE inter-node, falls back to sockets if misconfigured
- Multi-node rendezvous: rank 0 opens a TCP store, all ranks register, NCCL unique ID exchanged, then training starts
- The bandwidth ladder — NVLink > NVSwitch > InfiniBand > RoCE > PCIe > TCP — explains most scaling surprises
- Debug rendezvous with telnet MASTER\_ADDR PORT; debug transport selection with NCCL\_DEBUG=INFO